

Block Diffusion 에서의 Discriminator Importance-Weight Guidance

판별자의 조건부 형태와 유도(guidance) 수식 정식화

Block-Diffusion-OD-Planning (v2 backbones)

2026-07-21

Abstract

본 문서는 (i) block diffusion 상황에서 discriminator 가 무엇을 조건으로 보는지 —현재 블록만 보는가, 아니면 이전 블록(prefix)까지 보는가— 를 수식으로 규정하고, (ii) 그에 기반한 importance-weight guidance 를 두 커널(mask 흡수 커널, graph 균일 CTMC 커널)에 대해 모두 수식으로 명시한다. 각 수식에는 실제 구현 위치(models_seq/bd_models.py, models_seq/bd_disc.py)를 병기한다. 핵심 결론은: 판별자는 블록 단독이 아니라 **clean partial path** $[x^{<b} \parallel x_0^b]$ (이전 블록 전체 + 현재 블록)를 입력으로 받아야 하며 (Lemma 1), 블록만 보는 판별자는 편향된다 (Lemma 2).

1 표기와 Block Diffusion 세팅

경로를 canvas 형태 $x = [x^1, \dots, x^K]$ 로 두고, 위치들을 크기 B 의 블록 $x^1, \dots, x^{N_B}$ 로 나눈다($B = \text{block_size}$). 한 블록 b 를 디노이징할 때:

- $x^{<b}$: 이미 복원(reveal)된 이전 블록 전체 (clean prefix).
- x_t^b : 현재 블록의 시각 t latent, x_0^b : 그 clean 값.
- 디노이저 $p_\theta(x_0^b | x_t^b, x^{<b})$ 는 *offset-block-causal* attention (M_{OBC})을 통해 prefix $x^{<b}$ 를 조건으로 받는다 — 이것이 이전 블록 정보가 모델에 들어가는 유일한 경로이다.

두 within-block 커널:

- (mask) 흡수(absorbing) 상태 [MASK] 로의 독립 전이, SUBS 파라미터화, first-hitting 복원; (1)
- (graph) $Q_t = \exp((A - D)\beta_t)$, $\bar{Q}_t = \prod_{i \leq t} Q_i$, 균일분포를 극한분포로 갖는 D3PM CTMC. (2)

2 블록 조건부 목표분포와 밀도비

Guidance 의 목표는 모델 역과정 p_θ 대신 데이터 역과정 q 에서 블록을 복원하는 것이다. 블록 b 의 한 역스텝은

$$q(x_{t-1}^b | x_t^b, x^{<b}) = \sum_{x_0^b} q(x_{t-1}^b | x_t^b, x_0^b) q(x_0^b | x_t^b, x^{<b}) \quad (3)$$

$$= \mathbb{E}_{x_0^b \sim p_\theta(\cdot | x_t^b, x^{<b})} \left[w(x_0^b) q(x_{t-1}^b | x_t^b, x_0^b) \right] / \mathbb{E}[w], \quad (4)$$

여기서 후보별 importance weight 는 블록 조건부 밀도비

$$w(x_0^b) = \frac{q(x_0^b | x^{<b})}{p_\theta(x_0^b | x^{<b})} \approx \frac{D_\phi}{1 - D_\phi} \left([x^{<b} \parallel x_0^b] \right). \quad (5)$$

식 (4) 는 self-normalized importance sampling 이며, x_t^b 에만 의존하는 후보-공통 인수는 정규화에서 소거된다(아래 Lemma 1).

3 판별자는 무엇을 조건으로 보는가: prefix || block

Lemma 1 (Clean partial-path 판별자로 충분하다). 판별자를 부분 경로 $[x^{<b} \| x_0^b](prefix + \text{현재 clean 블록})$ 에서 “실제 데이터 vs 모델 표본”으로 학습하면 그 밀도비는

$$r(x^{<b} \| x_0^b) = \frac{q(x^{<b}, x_0^b)}{p_\theta(x^{<b}, x_0^b)} = \frac{D_\phi}{1 - D_\phi}. \quad (6)$$

우리가 필요한 블록 조건부 가중치 (5) 는

$$w(x_0^b) = \frac{q(x_0^b | x^{<b})}{p_\theta(x_0^b | x^{<b})} = \underbrace{\frac{q(x^{<b}, x_0^b)}{p_\theta(x^{<b}, x_0^b)}}_{=r} \cdot \underbrace{\frac{p_\theta(x^{<b})}{q(x^{<b})}}_{=C^{-1}} = C^{-1} r(x^{<b} \| x_0^b), \quad (7)$$

이고 $C = q(x^{<b})/p_\theta(x^{<b})$ 는 *prefix* 에만 의존하여 블록 b 안의 모든 후보 x_0^b 에 대해 상수이다. 따라서 *self-normalized* 가중치 $w_n / \sum_m w_m = r_n / \sum_m r_m$ 에서 C^{-1} 이 소거되어, **joint partial-path** 비율 $r = D_\phi / (1 - D_\phi)$ 를 그대로 써도 결과가 동일하다.

Lemma 2 (블록만 보는 판별자는 편향된다). 입력을 현재 블록 x_0^b 만으로 제한한 판별자는 주변 밀도비 $q(x_0^b)/p_\theta(x_0^b)$ 를 학습한다. 이는 목표 (5) 와

$$\frac{q(x_0^b | x^{<b})}{p_\theta(x_0^b | x^{<b})} = \frac{q(x_0^b)}{p_\theta(x_0^b)} \cdot \frac{q(x^{<b} | x_0^b)/q(x^{<b})}{p_\theta(x^{<b} | x_0^b)/p_\theta(x^{<b})} \quad (8)$$

만큼 다르며, 두 번째 인수는 후보 x_0^b 에 따라 변하므로 상수처럼 소거되지 않는다. 즉 블록만 보는 판별자는 *prefix* 와의 정합(연결성·방향)을 무시하는 편향된 가중치를 준다.

Remark 1 (구현). 판별자 `BDDiscriminator.forward` (`bd_disc.py:119`)는 `canvas` 형태 부분 경로 `tokens=[dst, ori, v1, ..., vj]` 를 `lengths` 로 잘라 입력받는다 (주석: “*canvas-form partial paths*”, `bd_disc.py:121`). 학습 표본은 `make_partial` (`bd_disc.py:163`) 이 한 경로에서 길이 $j \sim \mathcal{U}\{2, \dots, L\}$ 로 무작위 절단하여 만든다 — 즉 모든 길이의 *prefix* 에 대해 점수를 학습한다. 따라서 구현은 Lemma 1 의 partial-path 판별자이며, Lemma 2 의 block-only 는 아니다. Guidance 시점에도 점수는 *prefix*+현재 후보 블록 전체에 매겨진다: `full[:, :, -block:] = cand; disc(x_disc, ...)` (`bd_models.py:978--993`).

Lemma 3 (가변 길이 절단의 잔여 편향). 판별자를 무작위 절단 길이 $j \sim \mathcal{U}\{2, \dots, L\}$ (커널 $\kappa(\ell | L) = \frac{1}{L-1}$, `bd_disc.py:163`)로 학습하면, 길이 ℓ 인 부분 경로 y 에 대해 최적 판별자는

$$\frac{D_\phi}{1 - D_\phi}(y) = \underbrace{\frac{q(y)}{p_\theta(y)}}_{\text{원하는 prefix 주변비}} \cdot \underbrace{\frac{\Pr_q(\text{len}=\ell)}{\Pr_{p_\theta}(\text{len}=\ell)}}_{(*)} \cdot \underbrace{\frac{\mathbb{E}_{z \sim q(\cdot|y)}[\kappa(\ell | \ell + |z|)]}{\mathbb{E}_{z \sim p_\theta(\cdot|y)}[\kappa(\ell | \ell + |z|)]}}_{(**)} \quad (9)$$

을 추정한다($x = y \cdot z, z = \text{continuation}$). 한 블록의 모든 후보는 같은 길이 ℓ 로 절의되므로 $(*)$ 는 후보-공통이고 C 와 함께 *self-normalization* 에서 소거된다. $(**)$ 만 후보마다 다를 수 있는데, 이는 *prefix* y 를 잇는 실제/모델 *continuation* 의 남은 길이 분포 차이에서 온다. 고정 목적지 최단경로에서는 후보들이 인접 정점에서 끝나 *dst* 까지 남은 *hop* 분포가 유사하므로 $(**) \approx$ 후보-공통 $\Rightarrow$ 잔여 편향 미미. 정확한 불편화: 절의 길이로 절단을 고정하거나 표본을 $1/\kappa$ 로 재가중하면 $(**)$ 가 소멸한다.

요컨대 **prefix** 포함은 Lemma 1–2 로 필수·정확, 남는 것은 Lemma 3 의 절단 재가중 $(**)$ 뿐이며 본 세팅에서 무시 가능하다.

판별자 구조(요약). D_ϕ 는 (a) 시나리오 그래프 A_{scn} 위의 GraphSAGE 로 얻은 정점 임베딩, (b) 위치별 전이 피쳐 [edge-exists, is-transition, deg-ratio] (`bd_disc.py:136--146`; edge-existence 채널이 폐쇄된 edge 신호를 직접 노출하여 미학습 시나리오로 일반화), (c) 2-layer transformer + masked mean pooling (`bd_disc.py:152--154`), (d) bounded logit head 를 거친다:

$$\ell_\phi(x) = \lambda \tanh(z(x)/\lambda), \quad \frac{D_\phi}{1 - D_\phi} = e^{\ell_\phi} \in [e^{-\lambda}, e^\lambda], \quad \lambda = 4 \quad (\text{bd_disc.py:156}). \quad (10)$$

4 Guidance 수식 — 두 커널

두 커널 공통으로, 매 역스텝에서 N 개의 mean-field 후보를 뽑아 가중 평균 $\bar{x}_0^b$ 를 만들고 이를 posterior 에 대입한다. $\{x_0^{b,(n)}\}_{n=1}^N \sim p_\theta(\cdot | x_t^b, x^{<b})$,

$$w_n = \exp(\gamma \ell_\phi([x^{<b} \| x_0^{b,(n)}])), \quad \bar{x}_0^{b,l}(k) = \frac{\sum_n \mathbf{1}\{x_0^{b,(n),l} = k\} w_n}{\sum_n w_n} \quad (\text{bd_models.py:1003--1011, 1092--1099}). \quad (11)$$

($\gamma = \text{w_gamma}$ 는 tempering; $\gamma = 1$ 이면 논문 수식과 정확히 일치.)

4.1 Mask (흡수) 커널 — first-hitting

디노이저는 SUBS: $\hat{x}_0^b = \text{softmax}$ 로 각 마스크 위치의 정점분포를 준다. 매 reveal 에서 후보 (11) 로 $\bar{x}_0^b$ 를 만들고, 균일하게 고른 마스크 위치 하나를 $\bar{x}_0^b$ 에서 샘플하여 확정한다:

$$\ell^* \sim \mathcal{U}(\text{masked positions}), \quad x_0^{b,\ell^*} \sim \text{Cat}(\bar{x}_0^b \ell^*) \quad (\text{bd_models.py:1013--1023}). \quad (12)$$

4.2 Graph (균일 CTMC) 커널 — D3PM posterior

D3PM 역 posterior 에 clean 블록 대신 재가중 평균 $\bar{x}_0^b$ 를 대입한다:

$$p(x_{t-1}^b | x_t^b, x^{<b}) = \text{Cat}(x_{t-1}^b; p \propto x_t^b Q_t^\top \odot \bar{x}_0^b \bar{Q}_{t-1}) \quad (\text{bd_models.py:1103--1111}). \quad (13)$$

구현에서 `EtXt` = $x_t^b Q_t^\top$ (`bd\_models.py:1105`), `Em1` = $\bar{x}_0^b \bar{Q}_{t-1}$ (:1106), 곱 후 정규화 (:1107--1109). 이는 논문 Eq. (21)/(14) 와 정확히 일치한다.

4.3 Self-normalized 추정과 ESS

식 (4),(11) 의 추정 품질은 Kish effective sample size 로 진단한다:

$$\text{ESS} = \frac{(\sum_n w_n)^2}{\sum_n w_n^2} = \frac{1}{\sum_n \tilde{w}_n^2} \in [1, N], \quad \tilde{w}_n = \frac{w_n}{\sum_m w_m} \quad (\text{bd_models.py:1000, 1089}). \quad (14)$$

$\text{ESS} \rightarrow N$ = 가중치 균등 = 판별자가 후보를 구분 못 함(guidance 무력); $\text{ESS} \rightarrow 1$ = 소수 후보 집중(guidance 강함, 분산 위험).

4.4 Adjacency-masked proposal (Lemma 3)

Lemma 4 (인접성 마스크는 불편(unbiased) 분산감소). 후보 제안을 “이미 복원된 이웃 기준 A_{scn} -합법 전이”로 제한하면, 제외된 후보는 q 의 지지집합 밖($w = 0$)이고, 위치별 정규화상수 Z_ℓ 은 후보-공통이라 소거된다. 따라서 목표분포 (3)는 불편이며 분산만 감소한다.

구체적 마스크(mask 커널, reveal당). 블록 위치 ℓ 의 이미 공개된 좌/우 이웃을 u_L, u_R 라 하면, 후보 정점 k 의 legality 마스크는

$$m_\ell(k) = \left[u_L \text{ 공개} \Rightarrow A_{\text{scn}}(u_L, k) \right] \cdot \left[u_R \text{ 공개} \Rightarrow A_{\text{scn}}(k, u_R) \right], \quad p_{\text{cand}} \leftarrow \frac{p_{\text{cand}} \odot m_\ell}{\sum_k (p_{\text{cand}} \odot m_\ell)} \quad (15)$$

즉 후보 k 는 공개된 양쪽 이웃과 시나리오 $edge$ 로 연결될 때만 살아남고, 그 외에는 p_{cand} 에서 확률 0이 된다(제거·미존재 edge 사용을 원천 차단). 공개 안된 이웃 쪽은 제약 없음(마스크 1). 이 곱형 마스크가 Lemma 4의 “ A_{scn} -합법 전이” 제한을 구현한다.

구현: `plan_guided(..., adj_prop=True)`; mask 커널 reveal당 마스크링 `bd_models.py:929--956`; graph 최종 디코드의 시나리오 인접성 사용 `bd_models.py:1113--1121`.

5 수식 ↔ 코드 대응표

수식	코드 위치 (models_seq/)
디노이저 $p_\theta(x_0^b x_t^b, x^{<b})$ (prefix=M _{OBC})	<code>bd_models.py:_cfg_logits / attn</code>
후보 $x_0^{b,(n)} \sim p_\theta$	<code>bd_models.py:971, 1068</code>
판별자 점수 $[x^{<b} \ x_0^{b,(n)}]$	<code>bd_models.py:978--993, 1071--1085</code>
$w_n = \exp(\gamma \ell_\phi), \ell_\phi = \lambda \tanh(z/\lambda)$	<code>bd_models.py:997--998, 1086--1087; bd_disc.py:156</code>
재가중 평균 $\bar{x}_0^b$ (11)	<code>bd_models.py:1003--1011, 1092--1099</code>
mask reveal	<code>bd_models.py:1013--1023</code>
graph D3PM posterior (13)	<code>bd_models.py:1103--1111</code>
ESS $(\sum w)^2 / \sum w^2$	<code>bd_models.py:1000, 1089</code>
partial-path 판별자 입력 (Lemma 1)	<code>bd_disc.py:119--157</code>
무작위 절단 학습표본	<code>bd_disc.py:163--168</code>
adj-masked proposal (Lemma 4)	<code>bd_models.py:929--956 (mask), 1113--1121 (graph)</code>